Lecture 7 · Part I · Lecture notes

Ensembles and random forests

Dataset: Forest CoverType

How to use these notes

The primary text for this lecture is Géron, Chapter 6. These notes are not a replacement for it and do not repeat it: they are the lecture's own argument written out at length — the derivation done slowly, the numbers with the runs that produced them, and the reasoning between one slide and the next that a deck can only gesture at. Read the chapter for the method; read these for why the lecture makes the choices it does. Every figure quoted here is printed by this lecture's notebook, and the course checks that mechanically rather than trusting it.

Contents

1 The result we are carrying forward **1**

1.1 Where many different models come from 2

2 The mathematics: the variance of an average of correlated predictors **2**

2.1 Two, then n 2

2.2 Reading the two terms 3

3 Building them, and measuring ρ **3**

3.1 Voting, the simplest ensemble 4

3.2 Bagging, forests and extra-trees 4

3.3 A validation set you get for nothing 4

3.4 Measuring ρ needs something we do not have 5

4 And what did that do to the accuracy? **5**

5 The bill **6**

5.1 Feature importance, and a control 6

6 Boosting, and what practitioners reach for **7**

7 Did the averaging fix what we diagnosed? **8**

8 The notebook **8**

9 Exercises **8**

Summary **9**

---

1 The result we are carrying forward

Lecture 6 built a decision tree on this dataset: CART chose each split to minimise the size-weighted impurity of the two children, and the brief's eight conditions per prediction fixed the depth at 8. This lecture starts with a measurement rather than a score. Refit the same depth-8 tree, with the same hyperparameters and the same seed, on twenty different 90% subsamples of the same training rows: test accuracy moves by a standard deviation of 0.25 points, and *9.1% of the individual predictions change*. Only 71.4% of test patches get the same answer from all twenty.

A stable metric, an unstable model. In the language of Lecture 5's decomposition this is *variance*: sensitivity of the fitted function to the particular training sample. The whole of today follows from one question: what does averaging do to variance?

Forest CoverType	Test accuracy	Conditions per justification
Always "Lodgepole Pine"	48.8%	0
Depth-8 tree, minimum leaf 20	73.0%	7.97
Unconstrained tree	82.6%	17.88

1.1 Where many different models come from

Ask a difficult question of a thousand random people and aggregate their answers, and the aggregate is often better than an expert's. The same is true of models, and it has a name: an **ensemble**. A group of weak learners can form a strong learner — provided there are enough of them and they are sufficiently *diverse*. "Sufficiently diverse" is doing all the work in that sentence; it is the informal name of a quantity the next section defines.

Why this lecture follows the one on trees: averaging attacks variance, and trees are the high-variance model we have just measured; a tree is cheap to fit, so two hundred are affordable; and an unconstrained tree has *low bias* — which matters, because averaging does nothing whatever to bias, so the members had better not have any to spare. That third point is why the ensembles here use unconstrained members. It looks reckless after a lecture spent limiting depth, and by the end of the next section it will look obvious.

2 The mathematics: the variance of an average of correlated predictors

Forget trees for a moment. You have n predictors of the same quantity, each with variance σ^2, and you average them: $\bar{f} = \frac{1}{n}\sum_t f_t$. What is $\mathrm{Var}(\bar{f})$?

The answer everyone gives first is σ^2/n. That step is valid only when the f_t are *uncorrelated* — the variance of a sum is the sum of the variances *plus twice the sum of the covariances*. Dropping the covariances is an assumption, and it is exactly the assumption that fails for an ensemble of trees grown on the same dataset.

2.1 Two, then n

With $\mathrm{Var}(f_1) = \mathrm{Var}(f_2) = \sigma^2$ and $\mathrm{Corr}(f_1, f_2) = \rho$:

$$\mathrm{Var}\Big(\tfrac{f_1+f_2}{2}\Big) = \tfrac{1}{4}\big[\sigma^2 + \sigma^2 + 2\rho\sigma^2\big] = \frac{\sigma^2(1+\rho)}{2}.$$

At $\rho = 0$ this is the familiar halving; at $\rho = 1$ it is σ^2, because the two predictors are the same predictor and averaging achieves nothing. The general case has both limits in it.

n identically distributed predictors, common pairwise ρ

Assume each has variance σ^2 and $\mathrm{Cov}(f_s, f_t) = \rho\sigma^2$ for $s \neq t$. Then

$$\mathrm{Var}\Big(\sum_{t=1}^{n} f_t\Big) = \sum_{s=1}^{n}\sum_{t=1}^{n} \mathrm{Cov}(f_s, f_t),$$

a double sum of n^2 terms. Split them by whether $s = t$: n diagonal terms each σ^2, and $n(n-1)$ off-diagonal terms each $\rho\sigma^2$. Divide by n^2:

$$\mathrm{Var}(\bar{f}) = \frac{1}{n^2}\big[n\sigma^2 + n(n-1)\rho\sigma^2\big] = \frac{\sigma^2}{n} + \frac{n-1}{n}\rho\sigma^2,$$

and writing $\frac{n-1}{n} = 1 - \frac{1}{n}$ and gathering the $1/n$ terms,

$$\boxed{\mathrm{Var}(\bar{f}) = \rho\sigma^2 + \frac{(1-\rho)\sigma^2}{n}}$$

Identically distributed, not independent — that is the whole point. n trees grown on bootstrap samples of one dataset have the same distribution and are certainly not independent: they were handed the same dataset.

2.2 Reading the two terms

The sentence to remember

Averaging destroys the independent component of the variance and leaves the correlated one completely untouched.
And note what it does *not* say. It says nothing about bias: an average of n biased predictors is exactly as biased as one of them.

As n grows the second term vanishes, so the variance falls onto the floor $\rho\sigma^2$ that no number of members can touch.

So the way to improve an ensemble is *not more members*. It is less correlated members. And — the point that gets forgotten — anything which lowers ρ by making individual members worse is a trade, not a free lunch, because σ^2 appears in *both* terms and can rise while ρ falls.

One formula, three algorithms

Bagging randomises the rows. A random forest also randomises the columns. Extra-trees also randomises the thresholds. All three exist for one purpose: to lower ρ, because $\rho\sigma^2$ is the floor that n cannot reach below.

3 Building them, and measuring ρ

3.1 Voting, the simplest ensemble

A `VotingClassifier` combines different algorithms fitted to the same data — in the deck's example a tree, a logistic regression and a 5-nearest- neighbour classifier. `voting="hard"` takes the majority of the predicted labels; `voting="soft"` averages the predicted probabilities, and is usually better, because a confident member outweighs several hesitant ones. Different algorithms make different kinds of errors, which is the informal way of saying ρ is small. The formula applies unchanged.

3.2 Bagging, forests and extra-trees

Bagging (*b*ootstrap *agg*regat*ing*) draws m rows with replacement, fits a tree, and repeats n times. Each bootstrap sample has the size of the training set but only about 63% of its distinct rows; the ensemble aggregates by majority vote for classification and by the mean for regression. Each member is individually worse than one tree fitted to the whole set, and the ensemble is better — that is the formula, not a paradox. The members are unconstrained trees, because averaging cannot remove bias and is about to remove the variance. **Pasting** is the same without replacement, producing less diverse members, so bagging usually wins — but it is a cheap thing to cross-validate rather than assume.

A **random forest** adds a random subset of features at every node — the classification default `max_features="sqrt"` gives $\lfloor\sqrt{54}\rfloor = 7$ of our 54 columns, drawn afresh at each node, so a tree with 200 nodes draws 200 different subsets. **Extra-trees** goes one step further: having chosen the candidate features, draw one threshold at random for each and keep the best of those, rather than searching all thresholds. That is much faster, because the threshold search is the expensive part

of training a tree, and it makes members more diverse. Holding the rows fixed it also makes each member worse — on bootstrap samples, 72.8% against the forest member's 73.6%. Note that Scikit-Learn's extra-trees default is `bootstrap=False`, so the random thresholds *replace* the bootstrap rather than joining it; a default extra-trees member sees every training row, and on this dataset comes out *better* than a forest member, 77.8% against 73.6%.

3.3 A validation set you get for nothing

Each bagged member never sees about 37% of the rows — its **out-of-bag** instances. A given row is missed by one draw with probability $1 - 1/m$, so by all m draws with probability

$$\left(1 - \frac{1}{m}\right)^m \longrightarrow e^{-1} \approx 0.368,$$

hence about 63% distinct rows in and 37% held out. Score each member on its own out-of-bag rows and you have a generalisation estimate with no validation split and no extra fit.

Read it carefully, though. It is an average over members, each scored on a different 37%, so it estimates the generalisation of one member's *procedure*, not of the ensemble you ship. It is available only where members see bootstrap samples: with `bootstrap=False`, the extra-trees default, there are no out-of-bag rows and `oob_score=True` raises. Use it to compare configurations cheaply, then confirm the one you pick with cross-validation or the test set. It does not replace the test set — it replaces the validation set.

A leakage detector you get free

If the out-of-bag score comes out much *better* than the test score, something in the pipeline saw the out-of-bag rows anyway — a scaler fitted outside the ensemble, or a feature built from the whole frame.

3.4 Measuring ρ needs something we do not have

ρ is the correlation between two members over the randomness of the whole procedure, which includes *which training set you were handed*. Two trees grown on one dataset are correlated through that dataset — so a single training set cannot show it. Conditional on the data, the members are independent by construction and ρ would measure as zero.

CoverType is large enough to cut 20 *disjoint* training sets of 20,000 rows each, no row reused, and still keep 12,000 aside as a shared test set, then fit a 20-member ensemble on each. Measured that way, ρ is 0.072 for bagging, 0.043 for the random forest, 0.067 for extra-trees at its default and 0.043 for extra-trees with a bootstrap. Every variant is below bagging, and the measured variance of an average of n members falls on the curve $\rho\sigma^2 + (1 - \rho)\sigma^2/n$ for all four, with nothing fitted to make it.

4 And what did that do to the accuracy?

200 members, 48,000 training patches	10 members	200 members
Bagging	87.7%	89.8%
Extra-trees	86.7%	88.9%
Random forest	85.9%	88.4%
Extra-trees, `bootstrap=True`	85.2%	88.0%
The legible depth-8 tree	—	73.0%
Always "Lodgepole Pine"	—	48.8%

Every ensemble beats the constrained tree by at least fifteen points and the unconstrained tree by at least five. The last 190 members buy about two: most of the gain arrives by thirty members, steep then flat, exactly as $(1-\rho)\sigma^2/n$ says it should.

Now put the two tables beside each other:

Ensemble	ρ	One member's accuracy	Ensemble accuracy
Bagging	0.072	78.8%	89.8%
Extra-trees	0.067	77.8%	88.9%
Random forest	0.043	73.6%	88.4%
Extra-trees, boot	0.043	72.8%	88.0%

The ensemble with the highest ρ is the most accurate.

That is not a contradiction — read the formula again

The formula is about *variance*. It says nothing about bias, and accuracy depends on both. Every mechanism that lowers ρ does so by *handicapping the members*. A tree allowed only 7 of 54 columns per node is a worse tree: 73.6% against bagging's 78.8%. And σ^2 appears in both terms, so lowering ρ while raising σ^2 can leave you worse off.

5 The bill

Failure condition — an ensemble has no per-prediction justification

The 200 trees of the forest hold **1,497,919 leaves** between them. The justification for one prediction is now 200 decision paths and a vote. Drawn to scale, against leaves on a logarithmic axis: the legible depth-8 tree scores 73.0%, the previous lecture's unconstrained tree 82.6%, and the forest at 1.5 million leaves 88.4% — fifteen points of accuracy, and four orders of magnitude more model to read.

On the vote share: a single tree's `predict_proba` returns the class proportions of one leaf, piecewise constant and often built on a handful of rows. An ensemble averages 200 such vectors from 200 different leaves, so the result varies smoothly and is better calibrated — one thing the agency asked for that averaging improves rather than destroys. But better calibrated is not calibrated: a forest's vote share is systematically pulled towards the middle, because unanimity is rare. If a

downstream decision uses the number as a probability, check it against observed frequencies before believing it.

5.1 Feature importance, and a control

`feature_importances_` sums, over every node that split on a feature, the weighted impurity reduction that split achieved — the ΔI of Lecture 6, accumulated. It runs, it is fast, and the top of the list is entirely sensible: elevation first, then the distances. Nothing invites suspicion.

Before trusting any ranking

Ask what it would look like on a variable you know carries no information. If you cannot answer, add one and find out.

Add a column of uniform random numbers and refit. It ranks **tenth of fifty-five**.

The repair is to break the column and see what breaks: shuffle one column of the *held-out* set, re-score, and report a spread over repeats (`permutation_importance`). It goes beyond Chapter 6 and is beyond the syllabus, for context; it is here because the alternative is to ship the biased measurement.

On the same model and the same columns, permutation importance on held-out data puts the decoy indistinguishable from zero, where impurity importance put it tenth.

6 Boosting, and what practitioners reach for

Everything so far trains members in parallel and averages them. **Boosting** trains them in *sequence*, each correcting its predecessor. The members are not identically distributed and are certainly not exchangeable, so *today's formula does not apply to boosting*. Bagging reduces variance; boosting reduces bias. Two consequences follow from the sequence: boosting cannot be parallelised across members, and it is much easier to overfit with.

AdaBoost fits a weak learner — typically a tree of depth 1, a decision stump — increases the weight of the instances it got wrong, fits the next learner on the reweighted data, and repeats; it predicts by a weighted vote, each learner weighted by its own accuracy. If it overfits, reduce `n_estimators` or `learning_rate`, or regularise the base estimator.

Gradient boosting keeps the sequence but changes the correction: instead of reweighting the instances, each new learner is fitted to the *residual errors* of the ensemble so far. `learning_rate` scales each member's contribution; a low rate needs more members and generalises better, which is called **shrinkage**. Too few members underfits and too many overfits, so use early stopping (`n_iter_no_change`): the number of members is not a "more is better" parameter, which is exactly the opposite of bagging.

Failure condition — the intuition that does not carry across

Adding trees to a bagged ensemble can only help, because $(1-\rho)\sigma^2/n$ falls monotonically. Adding trees to a *boosted* ensemble eventually hurts. Carrying the first intuition into the second is one of the commonest mistakes with these models.

`HistGradientBoostingClassifier` bins each feature into at most 255 buckets before growing anything,

so a split search costs $O(\text{bins})$ rather than $O(m)$. It is dramatically faster on a dataset this size, it is the algorithm LightGBM and XGBoost implement, it handles missing values and categorical columns natively — no imputer, no encoder, provided a categorical column has at most `max_bins` (by default 255) categories, or it raises an error — and on tabular data it is the usual first choice and on most benchmarks the strongest single model. On this dataset, at a budget that fits a lecture, it lands several points below the bagged forest — usual is not always.

Stacking trains a model to do the aggregation, with the base learners' predictions as the input features of a *blender*. The blender must be trained on predictions the base learners made for instances they did not see; `StackingClassifier` does this with cross-validation internally, which is the only reason it is safe to use. Build it by hand and the blender sees in-sample predictions, learns to trust whichever base learner overfits hardest, and the ensemble comes out worse than its best member — with no error and no warning.

Situation	Reach for
A high-variance base learner and time to spare	bagging or a random forest
Many weakly informative columns	a random forest — feature subsampling earns its keep
Training time is the binding constraint	extra-trees
You want the strongest tabular model	histogram gradient boosting — usually, and measure it against a forest anyway
You must justify individual predictions	none of them — go back to one tree

The last row is what this pair of lectures exists to teach, and it is the row people skip.

7 Did the averaging fix what we diagnosed?

The diagnosis was *instability*, not inaccuracy — so test the thing that was diagnosed, not the thing that is easy to measure. On 20 disjoint training sets, a random forest's variance of one member's correctness is 0.16734 and of the average of 20 is 0.01508: 91.0% of the variance removed, against a floor of 0.00721 it cannot reach below. Nine tenths of a single member's variance, gone — and we knew before starting both that it would stop and roughly where. The notebook repeats the twenty-subsample disagreement measurement of the first section on a forest; predict the answer before you run it.

8 The notebook

What to change

Set `max_features=None` on the random forest, which turns it back into bagging. Watch ρ rise, each member get better, and the ensemble accuracy rise with it — the trade of the whole lecture, in one keyword.

9 Exercises

The five set at the end of this lecture's deck, in the style of the written paper. The solutions are on Lecture 8's deck and in the exercise book.

1. For n predictors each of variance σ^2 and pairwise correlation ρ, the variance of their average is $\rho\sigma^2 + \frac{1-\rho}{n}\sigma^2$. State the limit as $n \to \infty$, and what it means for ensembles. [5]

2. Bagging measures $\rho = 0.072$ with one member at 78.8% accuracy; the random forest measures $\rho = 0.043$ with one member at 73.6%. Yet bagging's ensemble is the more accurate. Explain, using the formula above. [5]

3. You set `bootstrap=False` and `oob_score=True`. State what happens and why. [3]

4. Impurity importance ranks a column of random numbers tenth of 55; permutation importance on held-out data puts it at zero. Describe how permutation importance is computed, and why a random column cannot score well on it. [5]

5. A forest improves accuracy over one tree by far less than the variance reduction would suggest. Is this a contradiction? Answer with a reason. [4]

Summary

A depth-8 tree refitted on twenty 90% subsamples moved its accuracy by 0.25 points while 9.1% of its predictions changed: variance. From the double sum of covariances, $\mathrm{Var}(\bar{f}) = \rho\sigma^2 + (1-\rho)\sigma^2/n$: averaging destroys the independent component of the variance and cannot touch the correlated one, so the way to improve an ensemble is less correlated members, not more of them.

Bagging randomises the rows, a random forest also the columns, extra-trees also the thresholds. Measured on 20 disjoint training sets, ρ fell from 0.072 for bagging to 0.043 for the forest, with the measured variances on the curves the formula predicts. The lowest-ρ ensemble was not the most accurate, because every mechanism that lowers ρ also weakens the members and σ^2 sits in both terms. About fifteen points of accuracy cost 1,497,919 leaves and the per-prediction justification the previous lecture was built around. Impurity importance ranked a column of pure noise 10th of 55; a permutation measurement on held-out data put it at zero. Boosting trains its members in sequence and reduces bias, so the formula does not apply to it.